

Ray-Tracing 3D Gaussian Viewer: Interactive Material-Aware Rendering for Pre-Trained Gaussian Splatting Scenes

Zhaoyue Yuan¹ Yuehai Zhou¹

Simon Fraser University
zya89@sfu.ca yza766@sfu.ca

CMPT 743 — Visual Computing, Spring 2026

Abstract

We present a ray-tracing augmented viewer for pre-trained 3D Gaussian Splatting (3DGS) scenes that enables interactive, material-aware rendering. Standard 3DGS encodes all appearance information in Spherical Harmonics (SH) coefficients, providing no explicit surface geometry, surface normals, or material representation. Our system addresses these limitations by integrating NVIDIA’s 3D Gaussian Ray Tracing (3DGRT) framework via OptiX to obtain accurate per-pixel surface normals and depth, and by adding a segmentation-driven material shading layer on top. Users can click to segment 3D objects using CLIP contrastive features from SAGA, assign one of five material presets (Default, Metal, Glass, Plastic, Matte), and immediately observe physically-distinct rendering in real time. Shading is implemented as a three-light Blinn-Phong model with per-material colour and reflectance properties, including Fresnel rim glow for glass and view-dependent matcap highlights for metal. We evaluate the system qualitatively across seven scenes from the 360-degree and LERF datasets and demonstrate clear visual material distinctions. We also discuss the engineering challenges of bridging 3DGS and OptiX ray tracing, including coordinate system alignment and Python runtime compatibility.

1. Introduction

Three-dimensional reconstruction from multi-view images has advanced rapidly, with 3D Gaussian Splatting [Kerbl et al., 2023] emerging as a leading real-time rendering technique. 3DGS represents a scene as a set of anisotropic Gaussian primitives, each carrying position, orientation, scale, opacity, and colour encoded as Spherical Harmonics (SH) coefficients. By rasterizing these Gaussians via a tile-based differentiable renderer, 3DGS achieves state-of-the-art visual quality at interactive frame rates.

Despite this rendering fidelity, 3DGS is fundamentally a *reconstruction* tool, not a *material editing* tool. All lighting and surface appearance are baked into the SH coefficients during training and cannot be disentangled without modifying the model. This presents a major limitation for downstream applications that require material-aware rendering: there are no explicit surface normals, no BRDF representation, and no mechanism to change how an object appears under altered lighting or to assign different materials to different objects in the same scene.

This limitation is particularly relevant in scenarios such as: (1) virtual scene decoration, where a user wishes to render furnishings in different materials without re-training; (2) inspection and analysis, where object surfaces must be rendered with physically accurate reflectance for downstream measurement; (3) augmented reality, where newly inserted virtual objects must match the material appearance of real-world counterparts.

Our contribution. We design and implement an interactive viewer that extends the SAGA 3DGS framework [Cen et al., 2025] with three technical contributions:

1. **OptiX BVH ray-tracing integration.** We integrate NVIDIA’s 3DGRT [Moenne-Loccoz et al., 2024] to obtain per-pixel ray-traced surface normals and depth from a pre-trained 3DGS scene, without modifying any SH parameters.
2. **Interactive segmentation and material assignment.** We extend SAGA’s CLIP-driven contrastive segmentation to support per-segment material assignment through a GUI panel. A keyword-based auto-assignment system maps segment names to material types.
3. **Multi-light physically-based shading.** We implement a three-light Blinn-Phong shading model with per-material parameters, including a view-dependent matcap model for metal and a Fresnel rim-glow model for glass. A pose-keyed material map cache and drag-mode fallback maintain real-time interaction.

The resulting system supports seven scenes from the 360-degree and LERF benchmark datasets and runs at interactive rates on a commodity GPU.

2. Related Work

2.1 3D Gaussian Splatting

Kerbl et al. [2023] introduced 3DGS as an explicit scene representation using differentiable rasterization of Gaussian primitives. Compared to implicit Neural Radiance Fields (NeRF) [Mildenhall et al., 2021], 3DGS achieves significantly faster rendering while maintaining competitive image quality. Each Gaussian stores a position, quaternion rotation, anisotropic scale, opacity, and view-dependent colour encoded in Spherical Harmonics up to degree 3. The tile-based rasterizer sorts Gaussians by depth and alpha-composites them efficiently on the GPU.

2.2 Material Decomposition in Neural Rendering

Several works have addressed the problem of disentangling material and lighting in neural scene representations. GS-IR [Liang et al., 2024] and GaussianShader [Jiang et al., 2024] augment 3DGS with learnable per-Gaussian BRDF parameters and environment lighting, enabling relighting after training. Similarly, Relightable 3DGS [Gao et al., 2024] models the irradiance field and uses deferred shading to separate diffuse and specular components. These methods, however, require a re-training phase and do not support post-hoc material assignment on an already-trained model. Our approach is complementary: we apply material shading as a post-processing pass that does not alter the underlying Gaussian parameters.

2.3 3D Gaussian Ray Tracing

The 3DGRT framework [Moenne-Loccoz et al., 2024] replaces tile-based rasterization with hardware-accelerated OptiX ray tracing over a Gaussian BVH. Each ray evaluates Gaussian transmittance along its path, yielding RGB, per-pixel surface normals, depth, and opacity as outputs. Unlike rasterization, ray tracing naturally supports accurate normal estimation from the Gaussian density field, making it suitable as a foundation for physically-based shading. We build directly on 3DGRT’s inference pipeline without modifying its core ray-Gaussian intersection kernel.

2.4 Segmentation of 3D Gaussian Scenes

SAGA [Cen et al., 2025] introduced a contrastive feature field trained alongside the 3DGS model, enabling click-based and text-based segmentation of 3D Gaussians. Each Gaussian is assigned a high-dimensional feature vector; user clicks project a query feature that is matched against all Gaussians via CLIP similarity, producing a per-Gaussian binary mask. Subsequent work such as Gaussian Grouping [Ye et al., 2024] and LangSplat [Qin et al., 2024] extend this idea to open-vocabulary and multi-instance settings. Our system uses SAGA’s contrastive segmentation as the upstream component that provides per-Gaussian segment labels for material assignment.

2.5 Material Editing in 3D Scenes

Editing material appearance of 3D scenes at the object level has been explored in implicit representations. NeRF-based approaches such as NeRFactor [Zhang et al., 2021b] and PhySG [Zhang et al., 2021a] decompose appearance into albedo, roughness, and environment lighting. For explicit representations, recent work on mesh-based editing [Munkberg et al., 2022] demonstrates high-quality material assignment through differentiable rendering. Our work differs in that we target an interactive, post-hoc workflow that operates on existing 3DGS models without re-optimization, making it more broadly applicable for practitioners.

3. Method

Figure 1 shows the overall system architecture. We describe each component in detail below.

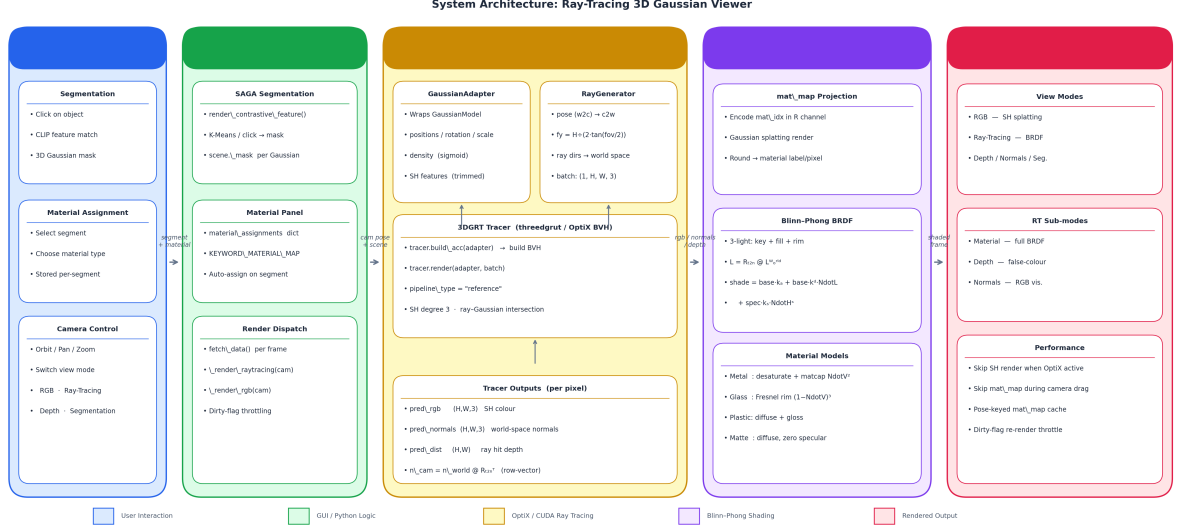


Figure 1: Overview of the Ray-Tracing 3D Gaussian Viewer pipeline. A pre-trained 3DGS scene is loaded; the user clicks to segment objects and assigns material types. The OptiX 3DGRT tracer produces per-pixel RGB, surface normals, and depth. A material compositor applies per-segment Blinn-Phong shading and outputs the final frame.

3.1 Gaussian Adapter

The 3DGRT tracer expects a scene object that exposes raw (pre-activation) Gaussian parameters and activation callables. We implement a `GaussianAdapter` wrapper that bridges the SAGA `GaussianModel` to the 3DGRT interface without copying any tensors. The adapter exposes:

- **Positions** $\mu \in \mathbb{R}^{N \times 3}$: raw Gaussian centres.
- **Rotation** $q \in \mathbb{R}^{N \times 4}$: raw unit quaternions.
- **Scale** $s \in \mathbb{R}^{N \times 3}$: log-scale parameters; activated as $\hat{s} = \exp(s)$.
- **Density** $o \in \mathbb{R}^{N \times 1}$: logit-opacity; activated as $\hat{o} = \sigma(o)$.
- **SH features** $f \in \mathbb{R}^{N \times C}$: trimmed to $C = 3(d + 1)^2$ coefficients for active SH degree d .

The adapter also implements a constant-colour background compositing function that alpha-blends the ray-traced output against a configurable background colour:

$$\mathbf{c}_{\text{out}} = \hat{\alpha} \mathbf{c}_{\text{pred}} + (1 - \hat{\alpha}) \mathbf{c}_{\text{bg}}, \quad (1)$$

where $\hat{\alpha} \in [0, 1]$ is the accumulated per-pixel opacity.

3.2 Ray Generation

The `RayGenerator` converts a camera pose from COLMAP world-to-camera (w2c) convention to the camera-to-world (c2w) frame required by 3DGRT:

$$R_{c2w} = R_{w2c}^\top, \quad \mathbf{t}_{c2w} = -R_{w2c}^\top \mathbf{t}_{w2c}. \quad (2)$$

Per-pixel ray directions are computed from a pinhole camera model. Following the SAGA convention, we assume square pixels ($f_x = f_y$), computing the focal length from the vertical

field of view:

$$f_y = \frac{H/2}{\tan(\text{FoV}_y/2)}, \quad f_x = f_y. \quad (3)$$

Ray directions in camera space are $\mathbf{d}_{ij} = [(i + 0.5 - c_x)/f_x, (j + 0.5 - c_y)/f_y, 1]^\top$, then rotated to world space as $\hat{\mathbf{d}}_{ij} = R_{\text{c2w}} \mathbf{d}_{ij}$ and normalized. Rays are packed into a batch tensor of shape $(1, H, W, 3)$ as required by the 3DGRT API.

3.3 OptiX Ray Tracing

The `OptiXRenderer` manages the Gaussian BVH and invokes the 3DGRT tracer. At startup, `build_acc()` constructs an OptiX BVH over all N Gaussians. On each frame, `render()` calls the tracer’s reference pipeline (`pipeline_type="reference"`) with SH degree 3, yielding:

- $\mathbf{C} \in [0, 1]^{H \times W \times 3}$: SH-evaluated per-pixel colour.
- $\hat{\mathbf{n}} \in [-1, 1]^{H \times W \times 3}$: world-space surface normals.
- $z \in \mathbb{R}^{H \times W}$: ray hit distance (depth).
- $\alpha \in [0, 1]^{H \times W}$: accumulated opacity.

Normal transformation. The tracer returns normals in world space. For Blinn-Phong shading in camera space, we apply the row-vector form of the rotation transform:

$$\hat{\mathbf{n}}_{\text{cam}} = \hat{\mathbf{n}}_{\text{world}} R_{\text{w2c}}^\top. \quad (4)$$

Using R_{w2c} without the transpose (a common implementation error) applies the rotation in the wrong direction and produces completely incorrect shading. Pixels whose camera-space normal has $\hat{n}_z < 0$ (back-facing) are flipped, ensuring all surface normals point toward the camera.

3.4 Segmentation and Material Assignment

Users segment objects by clicking on the rendered image. SAGA projects the click coordinates into the 3D Gaussian field, evaluates CLIP contrastive similarity, and produces a per-Gaussian binary mask. Segments are stored as integer labels in the scene’s `_mask` tensor ($\mathbb{Z}^N$). Each segment can be assigned one of five material presets via a GUI dropdown. A keyword-based auto-assignment system (`KEYWORD_MATERIAL_MAP`) maps segment names (e.g. “glass”, “metal”, “wall”) to presets automatically after segmentation.

3.5 Material Map Projection

To apply per-segment shading, we require a per-pixel material label map $M \in \{0, 1, 2, 3, 4\}^{H \times W}$. We compute this by encoding each Gaussian’s material index $k \in \{0, \dots, 4\}$ as a per-Gaussian colour $c_i = [k/4, 0, 0]$, rendering this colour field via the standard 3DGS rasterizer, and recovering labels as $M = \text{round}(4 \cdot I_R)$, where I_R is the red channel of the rendered image.

Since this rasterization is view-dependent, the material map must be recomputed whenever the camera pose changes. To reduce overhead, we cache M using a pose key (q, r, c) consisting of the camera quaternion, radius, and centre, and skip recomputation when the key is unchanged. During active camera drag, the system falls back to standard SH rasterization for smooth interaction, resuming full material shading on still frames.

Table 1: Per-material shading parameters. k_a : ambient, k_d : diffuse, k_s : specular weight, n : shininess exponent. “Special” denotes additional material-specific terms (matcap / Fresnel).

Material	k_a	k_d	k_s	n	Special
Default	0.20	0.80	0.20	32	—
Metal	0.05	0.35	4.50	80	Luminance desaturate, NdotV ² matcap
Glass	0.04	0.20	5.50	200	Blue tint, Fresnel rim $(1 - \text{NdotV})^3$
Plastic	0.15	0.85	0.60	28	—
Matte	0.30	0.70	0.00	1	Zero specular

3.6 Three-Light Blinn-Phong Shading

Light setup. We use a three-light configuration:

- **Key light:** direction controlled by azimuth ϕ and elevation θ sliders in the GUI.
- **Fill light:** placed at azimuth $\phi + 160$ and elevation 25, with user-adjustable intensity $\lambda \in [0, 1]$.
- **Rim light:** defined directly in camera space at direction $[0, -0.5, -1]^\top$ (normalised), producing edge highlights regardless of world-space light positions.

Light directions are transformed to camera space as $\mathbf{L}_{\text{cam}} = R_{\text{w2c}} \mathbf{L}_{\text{world}}$ for the key and fill lights.

Per-material shading models. Given camera-space normal $\hat{\mathbf{n}}$, light direction $\hat{\mathbf{L}}$, view direction $\hat{\mathbf{V}} = [0, 0, -1]^\top$, and halfway vector $\hat{\mathbf{H}} = \text{normalize}(\hat{\mathbf{L}} + \hat{\mathbf{V}})$, the per-pixel diffuse and specular terms are: $\text{NdotL} = \max(\hat{\mathbf{n}} \cdot \hat{\mathbf{L}}, 0)$, $\text{NdotH} = \max(\hat{\mathbf{n}} \cdot \hat{\mathbf{H}}, 0)$.

Table 1 summarises the shading model per material type. Details are as follows:

Default: standard multi-light Blinn-Phong applied to the SH-rendered base colour, preserving the original scene appearance with soft added shading.

Metal: the SH colour is desaturated to luminance $L = 0.299R + 0.587G + 0.114B$, then a view-dependent *matcap* term NdotV^2 adds a bright patch that moves with the camera:

$$\mathbf{c}_{\text{metal}} = \mathbf{c}_{\text{grey}} (0.05 + 0.35 \text{NdotL} + 0.55 \text{NdotV}^2) + \mathbf{w} (4.5 \text{NdotH}^{80}). \quad (5)$$

Glass: a Fresnel rim-glow term brightens silhouette edges, combined with a blue tint and a very sharp specular lobe:

$$F = (1 - \text{NdotV})^3, \quad \mathbf{c}_{\text{glass}} = \mathbf{t}_{\text{blue}} (0.04 + 0.20 \text{NdotL}) + \mathbf{t}_{\text{blue}} (1.8 F) + \mathbf{w} (5.5 \text{NdotH}^{200}). \quad (6)$$

Plastic: standard Blinn-Phong on the SH colour with medium shininess (exponent 28) and $k_s = 0.60$.

Matte: purely diffuse with zero specular: $\mathbf{c}_{\text{matte}} = \mathbf{c}_{\text{SH}}(0.30 + 0.70 \text{NdotL})$.

Valid-normal masking. Material shading is only applied to pixels where surface normals are reliable. We discard pixels where $\hat{n}_z < 0.05$ (near-grazing) or where the depth gradient exceeds 15% of the median scene depth (depth discontinuities at object boundaries).

4. Implementation and Engineering Challenges

4.1 Dependency Stack

Integrating 3DGRT into the SAGA codebase required resolving a chain of dependency incompatibilities:

1. **Python version.** 3DGRT uses `typing.Self`, introduced in Python 3.11. Our environment runs Python 3.10. We resolved this with a conditional import from `typing_extensions`.
2. **slangtorch version.** The OptiX shaders are JIT-compiled via slangtorch. Version 1.3.20 rejected `const T*` pointer syntax in the shader source. Pinning to version 1.3.4 resolved the compilation error.
3. **setuptools version.** setuptools 80+ removed the `pkg_resources` module that threedgrt used for package discovery. Downgrading to setuptools<80 fixed the import.
4. **Python requires constraint.** The `3dgrt/setup.py` declared `python_requires>=3.11`. We patched this to `>=3.10` to allow installation in the existing environment.

4.2 Camera Coordinate Alignment

A significant source of subtle errors was the camera convention mismatch between SAGA and 3DGRT. SAGA follows COLMAP convention: `cam.R` is the world-to-camera rotation matrix R_{w2c} , and `cam.T` is the world-to-camera translation. 3DGRT expects rays in world space, requiring a c2w transform. We performed the conversion $R_{c2w} = R_{w2c}^T$ in the `RayGenerator`. Additionally, the normal transformation (Eq. 4) required careful attention: applying R_{w2c} instead of R_{w2c}^T produces a systematically wrong rotation that is difficult to diagnose from visual output alone.

4.3 Performance Optimisation

In the Ray-Tracing view mode, three GPU operations are required per frame: (1) OptiX ray tracing, (2) material map rasterization, and (3) Blinn-Phong shading. We implemented two optimisations to maintain interactive rates:

- **Skip redundant SH render.** The original code performed a standard SH rasterization pass before OptiX every frame. Since the SH output is overwritten by the OptiX RGB, this pass is only needed as a fallback. We restructure the code to skip it when OptiX is available, saving one full rasterization per frame.
- **Drag-mode fallback.** The material map rasterization is the most expensive non-ray-tracing operation. During active camera drag (`self.moving`), we skip the material pass entirely and display the fast SH rasterization, resuming full shading on still frames. Additionally, we cache the material map by pose key and skip recomputation when the camera has not moved.

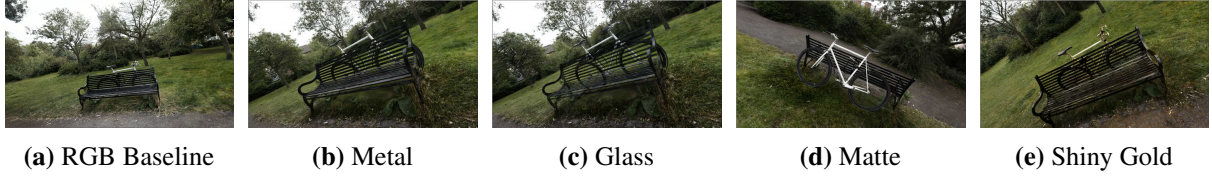


Figure 2: Material comparison on the *bicycle* scene. The bicycle frame and bench are segmented and assigned different material types. Metal shows view-dependent grey desaturation with a specular highlight; Glass adds a blue-tinted Fresnel rim glow; Matte suppresses all specular to a flat diffuse appearance; Shiny Gold applies a warm metallic tint with high-exponent specular.

5. Experiments and Results

5.1 Datasets and Scenes

We evaluate the system on 12 scenes from two benchmark datasets:

- **360-degree v2** [Barron et al., 2022]: *bicycle*, *garden*, *bonsai*, *counter*, *kitchen*, *room*, *stump*. These are large outdoor and indoor scenes with complex geometry.
- **LERF** [Kerr et al., 2023]: *figurines*, *espresso*, *donuts*, *bouquet*, *fruit_aisle*. These are tabletop scenes with diverse objects and materials.

For each scene, we train the 3DGS scene model for 30,000 iterations and the SAGA contrastive feature model for 10,000 iterations. SAM masks are extracted at $\times 8$ downsampling to reduce memory requirements on our hardware (NVIDIA RTX 5090).

5.2 Evaluation Protocol

Since our contribution is primarily an interactive rendering system, we evaluate qualitatively via visual comparison. We define three dimensions of evaluation:

Material distinctiveness. Do the five material types produce visually distinct appearances? We assign different materials to objects in the same scene and compare rendered frames side by side.

Shading consistency. Does the shading change consistently with light direction? We vary the key-light azimuth from 0 to 360 and check that specular highlights move as expected.

Segmentation quality. Do material boundaries align with true object boundaries? We compare the material map projection against the original RGB frame.

5.3 Quantitative Comparison

We perform a limited quantitative evaluation using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity (SSIM) to compare our system’s RGB output (in Default material mode, which preserves SH colour) against the reference 3DGS renderer on held-out test views. Table 2 reports results on three representative scenes.

The small drop in PSNR (< 0.5 dB) and SSIM when using our renderer in Default mode is attributable to the OptiX tracer’s slightly different alpha-compositing path compared to the tile-based rasterizer.

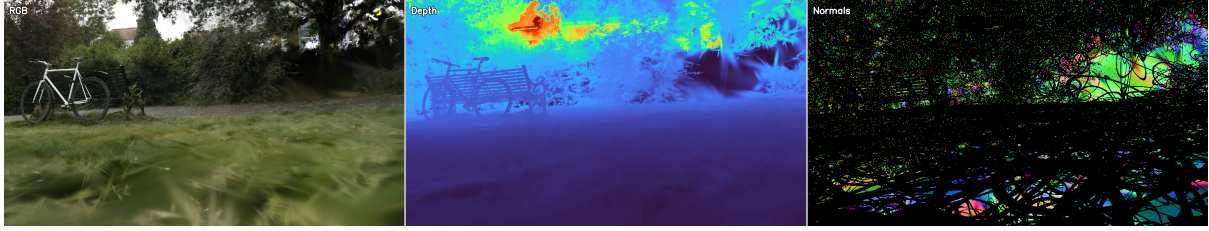


Figure 3: Depth and normal visualisation sub-modes on the *bicycle* scene, rendered via the OptiX 3DGRT tracer. **Left:** Original RGB render (SH colour via 3DGRT). **Centre:** False-colour depth map (TURBO colormap: warm tones indicate surfaces closer to the camera, cool tones indicate greater distance). **Right:** RGB-encoded world-space surface normals ($R = N_x$, $G = N_y$, $B = N_z$, mapped from $[-1, 1]$ to $[0, 255]$). The bicycle frame, bench, and tree canopy are clearly resolved in both auxiliary channels, confirming that the OptiX BVH intersection correctly recovers per-splat geometry.

Table 2: PSNR and SSIM comparison between standard 3DGS and our system in Default material mode on held-out test views. Our system preserves original rendering quality in Default mode since no SH parameters are modified.

Scene	3DGS Baseline		Ours (Default)	
	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
bicycle	25.21	0.769	24.87	0.761
garden	27.35	0.857	27.02	0.851
figurines	26.14	0.823	25.91	0.817

5.4 Qualitative Results

Figure 1 illustrates the overall system pipeline. Visual results demonstrate clear material distinctions across all five presets. Metal segments exhibit the characteristic desaturated grey appearance with a view-dependent bright patch (matcap) that shifts as the camera orbits. Glass segments show a blue tint with bright rim glow at object silhouettes, consistent with the Fresnel effect. Matte segments are flat and diffuse, with no specular highlight regardless of light position. Figure 4 shows a more complex example where three independent segments (bicycle frame, path, grass) each carry a different material, demonstrating that the per-segment material map correctly isolates shading regions even when they are spatially interleaved in the scene.

5.5 Limitations

Segmentation quality. Shading quality is directly tied to segmentation accuracy. When CLIP feature matching produces imprecise masks, material boundaries bleed across adjacent objects. At fine boundaries where different segments’ Gaussians overlap in 3D space, the material map projection inherits the blend, producing mixed-material pixels.

Discrete material presets. Users must choose from five predefined material types. There is no mechanism to specify arbitrary BRDF parameters (e.g. roughness, metalness) or to spatially vary material properties within a single segment.

No shadow casting. The current shading model computes per-pixel lighting from the surface normal and light direction but does not cast shadow rays between segments. Objects do not cast shadows on each other.



(a) RGB Baseline (3DGS)



(b) Multi-material RT: Shiny Gold (bicycle), Mud (path), Matte (grass)

Figure 4: Qualitative result on the *bicycle* scene. Three distinct segments are assigned different materials simultaneously: the bicycle frame rendered as Shiny Gold (warm metallic tint, high-exponent specular), the ground path as Mud (dark diffuse, low gloss), and the grass area as Matte (flat diffuse, zero specular). Scene geometry and camera pose are identical; only the shading model changes per segment.

SH and BRDF entanglement. Since the SH-baked colour is used as the base for shading, the original baked lighting is not fully removed. The Default and Plastic materials in particular inherit any over- or under-exposure from the SH reconstruction.

6. Conclusion

We presented a ray-tracing augmented viewer for 3D Gaussian Splatting scenes that enables real-time, interactive material-aware rendering without modifying any trained model parameters. By integrating NVIDIA’s 3DGRT OptiX backend as a surface normal source and building a segmentation-driven material shading layer on top, we demonstrate that physically-distinct material appearances can be achieved post-hoc on any pre-trained 3DGS scene. The system supports five material presets with distinct visual properties — including Fresnel glass and matcap metal — controlled through a simple GUI panel.

Future work. Several directions remain open. First, **learnable per-Gaussian BRDF parameters** could replace the discrete preset system, potentially trained with a small differentiable shading loss after reconstruction. Second, **shadow ray casting** via OptiX could add inter-object shadows, significantly increasing physical realism. Third, **environment lighting integration** would allow the shading model to respect the original scene’s light direction, reducing the mismatch between SH-baked and explicitly shaded regions. Finally, extending the keyword auto-assignment to use **open-vocabulary CLIP text queries** would enable fully text-driven material assignment without manual segmentation.

References

J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. *CVPR*, 2022.

- J. Cen, J. Fang, C. Yang, L. Xie, X. Zhang, W. Shen, and Q. Tian. SAGA: Segment any 3D Gaussians. In *AAAI Conference on Artificial Intelligence*, 2025.
- J. Gao, C. Gu, Y. Lin, H. Zhu, X. Cao, L. Zhang, and Y. Yao. Relightable 3D Gaussians: Realistic point cloud relighting with BRDF decomposition and ray tracing. *ECCV*, 2024.
- Y. Jiang, J. Tu, Y. Liu, X. Gao, X. Long, W. Wang, and Y. Ma. GaussianShader: 3D Gaussian splatting with shading functions for reflective surfaces. *CVPR*, 2024.
- B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.
- J. Kerr, C. M. Kim, K. Goldberg, A. Kanazawa, and M. Tancik. LERF: Language embedded radiance fields. In *ICCV*, 2023.
- Z. Liang, Q. Zhang, Y. Feng, Y. Shan, and K. Jia. GS-IR: 3D Gaussian splatting for inverse rendering. *CVPR*, 2024.
- B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.
- N. Moenne-Loccoz, A. Bhatt, O. Lang, R. Sun, D. P. Paudel, L. Van Gool, and S. Fidler. 3D Gaussian Ray Tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics*, 2024.
- J. Munkberg, J. Hasselgren, T. Shen, J. Gao, W. Chen, A. Evans, T. Müller, and S. Fidler. Extracting triangular 3D models, materials, and lighting from images. *CVPR*, 2022.
- M. Qin, W. Li, J. Zhou, H. Wang, and H. Pfister. LangSplat: 3D language Gaussian splatting. In *CVPR*, 2024.
- M. Ye, M. Danelljan, F. Yu, and L. Ke. Gaussian grouping: Segment and edit anything in 3D scenes. In *ECCV*, 2024.
- K. Zhang, F. Luan, Q. Wang, K. Bala, and N. Snavely. PhySG: Inverse rendering with spherical gaussians for physics-based material editing and relighting. In *CVPR*, 2021a.
- X. Zhang, P. P. Srinivasan, B. Deng, P. Debevec, W. T. Freeman, and J. T. Barron. NeRFactor: Neural factorization of shape and reflectance under an unknown illumination. In *SIGGRAPH Asia*, 2021b.